Radar Operacional

Documentação técnica do projeto

Versão de 07/10/2026 · funcionamento, decisões, estado atual e guia para retomar o trabalho

Este documento descreve o projeto como ele está hoje: o que faz, como foi construído, por que cada decisão foi tomada, o que já foi validado em ambiente real e o que ainda falta. Foi escrito para ser anexado a uma conversa com o assistente (Claude) ou entregue a quem for operar o sistema, de modo que o contexto não precise ser reconstruído. O passo a passo de instalação e operação está no **README.md** do repositório; aqui ficam a visão de conjunto e o raciocínio por trás do código.

Em uma frase	Um robô em Python que consulta o BigQuery, descobre quais CDs da GFW saíram para outro centro de trabalho (ou mudaram de situação) e avisa as pessoas certas por Teams e e-mail, sem repetir avisos e sem expor dado em claro.
Aplicação principal	Acompanhar as CDs abertas originadas na GFW que estão fora dela (tarefa `cds_gfw_fora_da_area`).
Natureza	Aviso auxiliar. Não substitui o SAP nem o controle oficial; em divergência, vale o sistema oficial.
Tecnologias	Python 3.12+, BigQuery, SQLite com cifragem por campo, webhook do Teams (Workflows), Outlook clássico (automação COM), Pillow e openpyxl para imagens e Excel, pytest (88 testes).
Repositório	github.com/Pedrocs50/radar-operacional (privado)

1. Contexto de negócio

O sistema atende quem acompanha as CDs da GFW no centro SJK1. Em vez de abrir o Power BI e procurar o que mudou, a pessoa recebe a informação pronta. A ideia resumida: o BI responde "como está?"; o Radar responde "o que mudou e o que precisa de atenção agora?".

Vocabulário

Termo	Significado
CD	Nota de não conformidade (tipo de nota `CD` no SAP QM), centro SJK1.
GFW e os 5 CTs	Pré-Junção, Junção, Pré-Pintura, Pintura Final E1 e Pintura Final E2. Os códigos estão no SQL e no YAML da tarefa.
CT de origem / CT atual	Origem = o CT da GFW que gerou a CD (classificação). Atual = onde a CD está agora. Uma CD está "fora da GFW" quando a origem é da GFW e o CT atual não é.
I0070 / I0072	Status do sistema da nota: I0070 = aberta; I0072 = encerrada.
TURNBACK	Indicador cujas premissas excluem certas CDs do acompanhamento (duplicadas, ferramental, efetividade, falta de material, protótipo, phase-out, decisão de programa, NFF, canibalização, aplicações especiais, logística/planning). O SQL principal já as aplica.
Medida	A tabela de origem tem várias linhas (medidas) por nota; o SQL mantém só a mais recente de cada uma.

As duas consultas

- **cds_gfw_fora_da_area** (foco do projeto): CDs abertas da GFW que estão fora da área, uma linha por nota, premissas do TURNBACK aplicadas. Traz cerca de 50 colunas (datas, responsáveis, plano, fluxo, material, PN, dias sem fluxo etc.).
- **cds_novas_do_dia**: CDs de classificação Escape, das cinco CTs da GFW, do dia mais recente da base. Serve para "novas do dia"; hoje só gera prévia local.
- Os SQLs originais **não são alterados**: qualquer ajuste fica no Python. Uma terceira consulta, `resolucao_cds.sql`, lê poucas colunas para explicar por que uma CD saiu da lista.

Tabela de origem: `opp_intermediate.consolidated_zqlrqm150_medidas` (cerca de 18,7 GB, sem partição). A consulta principal lê ~11,7 GB. Os dados nunca são mais atuais que a carga do SAP para o BigQuery; as mensagens mostram "dados de HH:MM".

2. Arquitetura

Tudo é uma **tarefa**: um arquivo YAML em `tasks/` que descreve fonte, mensagem, canais e os blocos opcionais (acompanhamento, sinais, gráficos, indicadores, resumo, saúde, e-mail). Uma nova tarefa é um novo YAML e um novo SQL. Fontes e canais seguem interfaces simples, então um canal novo (SharePoint, Graph) é um módulo novo sem mexer no motor.

Parte	Arquivos	Responsabilidade
CLI e configuração	radar/__main__.py, config.py, segredos.py	Comandos, leitura de settings e tarefas, segredos no Credential Manager.
Motor	engine/runner.py	Orquestra: fonte -> validação -> índice cego -> novos -> movimentos -> sinais -> mensagem -> canais -> estado -> log.
Análises	engine/movimentos.py, sinais.py, indicadores.py, paineis.py	Funções puras (sem I/O): movimentação, sinais de atenção, indicadores, contagens dos painéis.
Rotinas	engine/resumo.py, saude.py, doctor.py, simulacao.py	Resumo periódico, saúde do robô, diagnóstico do ambiente, exemplos de cartões.
Fontes	sources/bigquery.py, demo.py	Consulta, estimativa de custo, carimbo de atualização, resolução de notas. A demo usa dados fictícios.
Canais	channels/teams_webhook.py, email_outlook.py, email_conteudo.py, preview.py	Cartões do Teams, e-mail pelo Outlook, conteúdo do e-mail, prévia em HTML.

Parte	Arquivos	Responsabilidade
Imagens	channels/imagem.py, imagem_tabela.py, grafico.py	PNG por CD, tabela das CDs, gráfico semanal e painel de gráficos.
Armazenamento	storage/crypto.py, db.py	Cifragem Fernet e índice cego (HMAC); db.py é o único lugar com SQL.
Entorno	sql/, tasks/, config/, scripts/, templates/, tests/	Consultas, tarefas, configuração, scripts PowerShell, modelos HTML, testes.

O ciclo do vigia (a cada 5 minutos)

- Lê o carimbo da última carga da tabela de origem (metadado do BigQuery, **custo zero**). Se essa carga já foi tratada, termina.
- Se há carga nova, roda a consulta, registra os bytes cobrados e valida as colunas esperadas (se mudarem, falha com `SCHEMA_MUDOU` em vez de enviar algo errado).
- Transforma a chave de cada item (Nota_QM) em índice cego e separa os **novos** dos já avisados.
- Compara com a foto da execução anterior para achar CDs que saíram da lista ou mudaram de CT; para as que saíram, consulta a situação atual.
- Calcula sinais de atenção e vazamento por CT, monta o cartão do Teams e o e-mail, e entrega.
- **Só depois de entregar** grava o estado (já avisei, eventos, nova foto). Se algo falhar, a próxima verificação tenta de novo.
- Registra a verificação para a saúde do robô; falhas seguidas e retorno geram avisos.

3. Dados e segurança

Tabela (SQLite)	Conteúdo	Cifrado?
execucoes	Histórico: tarefa, início/fim, status, dry-run, contagens, bytes cobrados, código de erro.	Em claro (sem conteúdo)
alertas	Estado "já avisei": índice cego da chave + campos da CD (inclui CT de origem).	Payload cifrado
snapshots	Foto das CDs a cada execução real (campos configurados), base do "o que mudou".	Payload cifrado
eventos	Movimentação: resolvida, voltou para a GFW, mudou de CT, saiu das premissas, não localizada.	Payload cifrado
mensagens	Cópia da mensagem de cada execução (retenção de 90 dias).	Cifrado
verificacoes, estado, atualizacoes_tabela	Saúde do robô, chave/valor (último resumo, situação) e cargas detectadas da origem.	Em claro (sem conteúdo)

- **Cifragem por campo:** Fernet (AES + HMAC) com MultiFernet, que permite trocar a chave sem perder o histórico.
- **Índice cego:** a comparação "já avisei?" usa HMAC-SHA256 com chave própria (diferente da de cifragem); o banco compara códigos e nunca vê a nota.
- **Chaves e segredos** ficam no Credential Manager do Windows (serviço radar-operacional), nunca no código, no banco ou no git. O backup das chaves deve ser separado do banco: sem elas o histórico é irrecuperável.
- **Logs** trazem só códigos e tipos de erro, nunca conteúdo. A URL do webhook nunca aparece em log ou erro (URL inválida vira TEAMS_URL_INVALIDA).
- **Arquivos temporários** (prévias, imagens, Excel) contêm dado em claro: pasta `data/preview_tmp` com retenção de 24 h. A cifra por campo não os cobre; o servidor deve ter disco cifrado.
- **Sandbox:** só os canais de `sandbox.canais_permitidos` recebem envio real. Se ele bloquear todos os canais externos, o estado "já avisei" não é gravado, para o item sair quando o canal for liberado.
- **Dry-run por padrão:** só `--real` (ou o vigia) grava estado e envia.

4. Funcionalidades

4.1 Alerta de CD nova

Quando a tabela é carregada e aparece uma CD que saiu da GFW, o Teams recebe um cartão com as colunas principais (nota, modelo, serial, dias em aberto, CT de origem e atual, área, responsável, PN, problema, fluxo) e o e-mail recebe o estado geral com a CD nova em vermelho. Sem novidade, nenhuma mensagem. Rodar duas vezes não duplica avisos.

4.2 Acompanhamento: resolvida, voltou, mudou de CT

A cada execução real o robô guarda uma foto cifrada das CDs. Quando uma CD some da lista, consulta sua situação atual e classifica: **resolvida** (I0072), **voltou para a GFW** (CT atual entre os 5 CTs), **saiu das premissas do TURNBACK** (continua aberta fora da GFW, mas o filtro a excluiu) ou **não localizada**. Mudança de CT atual vira "mudou de CT". A primeira execução só cria a base de referência. Uma leitura vazia depois de um estoque com 3 CDs ou mais só é aceita na segunda vez seguida, para não anunciar "resolvidas" por causa de carga incompleta da origem.

4.3 Sinais de atenção

Sinal	Regra	Coluna
Parada	Sem movimento há 7 dias ou mais (configurável). Diferente de "aberta há X dias".	Dias_sem_Fluxo
Plano vencido	A data final planejada (com hora) já passou.	Data_Plan_Final, Hora_Plan_Final
Sem plano	Sem data final; datas vazias do SAP (0001-01-01, 9999-12-31) contam como ausentes.	Data_Plan_Final
Sem responsável	Campos de responsável da próxima ação vazios (ou 0).	Responsavel_Proxima_Acao, Nome_...
Vazamento por CT	Por CT de origem na GFW: abertas, %, média de dias, entradas em 7 e 30 dias, resolvidas em 30 dias.	Descricao_CT_Origem

Os sinais não geram alertas próprios: aparecem no resumo, no e-mail, no cartão do Teams e na coluna **Atenção** do Excel e da tabela (linha amarela).

4.4 Resumo periódico e gráficos

Agendado para **7h, 12h, 14h e 18h**: e-mail completo e cartão compacto no Teams, com o retrato do momento, o que se movimentou desde o último resumo, os pontos de atenção, o vazamento por CT, os resultados e a tabela de todas as CDs. **Só os resumos levam gráficos**, numa única imagem com quatro painéis (ranking em barras): onde as CDs estão agora (CT atual), CT de origem, tipo de defeito e responsável pela próxima ação. O "problema" em texto livre não agrupa, por isso o painel usa o tipo de defeito (trocável no YAML). O gráfico semanal de entraram x resolvidas fica numa seção própria. O cartão do Teams leva só o primeiro painel, em texto, porque imagem não cabe no webhook. O resumo só é marcado como feito depois de entregue; `--canais email:dono` envia só ao dono, sem marcar.

4.5 Indicadores de resultado

Entraram (CDs avisadas pela primeira vez, sem as da base de referência), resolvidas (resolvida ou voltou para a GFW), saldo e tempo médio para resolver (dias em aberto no momento da saída), em 7 e 30 dias e por semana (segunda a domingo; a corrente é parcial). Dependem de histórico acumulado: nos primeiros dias não há dado suficiente.

4.6 Saúde do robô

Cada chamada do vigia é registrada. Com 3 falhas seguidas (15 min), o dono recebe um e-mail com o código do erro e o log, e o grupo recebe um cartão simples ("problema técnico, já sendo verificado"). Ao voltar, os dois recebem o aviso de recuperação. Se passar mais de 30 minutos sem verificação (máquina desligada), só o dono é avisado quando o robô volta. Limite: um robô desligado não consegue avisar que está desligado; isso exige um monitor externo.

4.7 Ferramentas de apoio

- `doctor`: confere chaves, segredos, banco, BigQuery, Outlook, agendamentos e sandbox, sem enviar nada nem mostrar segredos.
- `simular`: cartões e e-mail de exemplo com dados inventados (visualizador em HTML); `--enviar` posta no canal de teste.
- `base-referencia`: grava o estoque atual como conhecido sem enviar nada (obrigatório antes do primeiro vigia real em uma máquina nova).
- `destinatarios`: mostra, adiciona ou remove e-mails de um destino sem apagar os demais; só aceita domínios corporativos (`email.dominios_permitidos`), e o envio também recusa outros domínios.
- `custo`, `historico` (GB por execução), `atualizacoes` (cadência das cargas), `status` (heartbeat).

5. Canais e mensagens

Canal	O que leva	Limites
Teams (webhook do Workflows)	Cartão adaptável com cabeçalho "RADAR OPERACIONAL". Tipos: alerta de CD nova, movimentação, resumo e sistema (falha/recuperação).	Cartão até ~24 KB (o Teams recusa ~28 KB; o motor reduz itens sozinho). Sem imagem. O remetente é o app Workflows; nome e foto próprios exigem um bot registrado.
E-mail (Outlook clássico)	Visão geral, pontos de atenção, vazamento, resultados, tabela em imagem (nova em vermelho, atenção em amarelo), Excel com todas as colunas e filtro, e gráficos nos resumos.	Sai em nome do usuário logado no Outlook. O endereço de e-mail do canal do Teams posta a mensagem dentro do canal. O "Novo Outlook" não tem a automação.
Prévia HTML	Cópia local do que seria enviado.	Dado em claro; retenção de 24 h.

Destinos usam o formato `tipo:nome` (por exemplo `teams:teste`, `email:dono`), e cada destino lê o segredo de mesmo nome (`teams_webhook_teste`, `email_destinatarios_dono`).

6. Operação

Quando	Comando	O que faz
Cada 5 min	scripts\vigiar.ps1 (vigiar)	Checa a carga da origem; se mudou, consulta e avisa.
7h, 12h, 14h, 18h	scripts\resumo.ps1 (resumo --real)	Resumo completo por e-mail e Teams.
Primeira vez	base-referencia	Registra o estoque atual sem enviar nada.
Qualquer hora	doctor, status, historico, atualizacoes, simular	Diagnóstico e acompanhamento.
Mudar quem recebe	destinatarios --adicionar / --remover	Edita a lista de e-mails de um destino; vale na próxima mensagem. Pessoas no Teams entram como membros do canal.

Implantação: clonar o repositório, criar o ambiente, fazer o login do BigQuery, guardar as chaves e os três segredos (URL do fluxo do Teams, destinatários de teste e e-mail do dono), validar com `doctor`, criar a base de referência e agendar com `scripts\agendar.ps1`. Os detalhes, incluindo migração de chaves e do banco, estão no README. As tarefas agendadas rodam como o usuário logado (usam o login do gcloud, o Credential Manager e o Outlook), então a sessão do servidor precisa ficar aberta.

Custo do BigQuery

Perguntar "a tabela mudou?" lê só o metadado (custo zero). A consulta de ~11,7 GB só roda quando a tabela é carregada de novo, então o custo é (cargas por dia) x 11,7 GB; o resultado em cache no mesmo dia pode custar zero. O ritmo real das cargas ainda não foi medido (as primeiras observações mostraram cargas com algumas horas de intervalo): `atualizacoes` registra a cadência. Há um teto de segurança por consulta (`bigquery.max_bytes_billed`, hoje 50 GB).

7. Decisões de projeto e preferências do dono

- **Escopo:** por enquanto só as duas consultas, com foco em `cds_gfw_fora_da_area`. O objetivo central é avisar "na hora" quando uma CD nova aparece fora da GFW.
- **Foco em mensagens, não em HTML:** o robô se comporta como um bot que manda Teams e e-mail; o HTML é só prévia.
- **E-mail e Teams têm papéis diferentes:** o Teams mostra a novidade (cartão curto); o e-mail mostra o estado geral e as tabelas.
- **Resumos nos horários 7h, 12h, 14h e 18h**, com gráficos só nesses relatórios.
- **Resolvida e mudou de CT** são importantes para mostrar resultado ao chefe, por isso há eventos persistidos e indicadores.
- **Fora do escopo por decisão do dono:** alerta de "dados atrasados" (a tabela é o veredito) e reaviso por idade (por enquanto).
- **MVP em nome do dono:** o e-mail sai da conta do dono pelo Outlook; uma caixa compartilhada (`email.remetente`) fica para depois. Itens que dependem da TI (conta de serviço, Graph, SharePoint, classificação da informação, co-proprietário do fluxo) foram adiados.
- **Dado operacional só em contas corporativas.** Nunca enviar CDs a e-mail pessoal: o envio e o comando destinatarios recusam domínios fora de `email.dominios_permitidos`. Destinatários ficam no Credential Manager, não no repositório.
- **Mensagens de teste** usam um canal só do dono; o sandbox permanece ativo até a TI classificar o que pode circular.
- **O dono prefere executar ele mesmo** o que envia mensagens ou publica código (commit e push), e quer explicações em português, diretas.

8. Estado atual (07/10/2026)

Validado em ambiente real

- Consulta no BigQuery conferida com o Power BI; estimativa de custo (~3,9 GB e ~11,7 GB).
- Cartão no canal de teste do Teams, e-mail com tabela em imagem e Excel, resumo por e-mail, vigia com carimbo de atualização, base de referência e anti-repetição com dado real, `doctor`.

Implementado e coberto por testes, ainda sem caso real

- Movimentação (resolvida, voltou, mudou de CT), saúde do robô, indicadores com histórico, sinais de "sem plano" e "sem responsável" (hoje todas as CDs têm plano e responsável), painel de gráficos com estoque grande.

Não existe ainda

- SharePoint, @menção no Teams, bot próprio, interface web, perfis por pessoa e segmentação por área, reaviso por idade, e-mail por Graph, servidor 24 h em operação.

Pendências com a TI

- Classificação da informação (o que pode circular por Teams, e-mail e SharePoint), antes de adicionar pessoas.
- Conta de serviço para o BigQuery, caixa compartilhada para o e-mail, co-proprietário do fluxo do Teams.
- Permissões do app registration (Mail.Send, Sites.Selected) e BitLocker no servidor (cobre prévias, imagens e Excel).

Próximos passos prováveis

- Colocar no servidor (README) e medir a cadência real das cargas e o custo.
- Segmentar por área; pasta do SharePoint com imagens e Excel; @menção.
- Análises de reincidência (mesmo PN ou defeito em várias CDs), ficha da CD, silenciar CD, horário de turno.

9. Guia para retomar o trabalho (para o assistente)

Convenções do código

- Código, comentários e mensagens em **português**; o dono lê e escreve em português.
- **Não alterar os SQLs originais** (`cds_gfw_fora_da_area.sql`, `cds_novas_do_dia.sql`); ajustes ficam no Python.
- SQL do SQLite só em `storage/db.py`. Análises em `engine/*` são funções puras (fáceis de testar). Nada de conteúdo em log.
- Qualquer envio passa pelo sandbox e só grava estado depois de entregar. Mudanças no motor exigem testes (`python -m pytest`, hoje 88 passando).
- Colunas novas usadas por um recurso devem entrar na validação de esquema (falha com SCHEMA_MUDOU), como já fazem sinais e painéis.

Onde mexer em cada tipo de pedido

Pedido	Onde
Novo sinal de atenção	engine/sinais.py (regra e tag), bloco sinais: do YAML, templates/email.html.j2 e _blocos_sinais em channels/teams_webhook.py.
Novo painel de gráfico	Só o bloco graficos: do YAML (titulo e campo); o código é genérico.
Novo canal (SharePoint, Graph)	Módulo novo em channels/, registrar em runner._resolver_enviador, incluir o destino em sandbox.canais_permitidos e nos canais da tarefa.
Nova tarefa	Novo YAML em tasks/ e novo SQL em sql/; reaproveita todo o motor.
Novo tipo de cartão	Função _corpo_* e entrada em _CORPOS em teams_webhook.py; exemplo em simulacao.py.
Mudar horário ou frequência	Parâmetros do scripts/agendar.ps1 (nada no código).

Armadilhas conhecidas

- **Windows e PowerShell 5.1:** scripts `.ps1` em UTF-8 precisam de BOM; `activate` pode ser bloqueado (usar .venv\Scripts\python); `&&` não existe.
- O e-mail usa a automação do Outlook clássico: precisa de usuário logado; pode mostrar aviso de segurança na primeira vez.
- **Colar segredos no terminal:** o `getpass` do Windows corrompe colagens repetidas; usar `--da-area-de-transferencia`. Se uma URL de webhook aparecer em tela ou conversa, tratá-la como exposta e recriar o fluxo.
- Cada consulta real custa ~11,7 GB; testes de desenvolvimento devem preferir dry-run ou o cache do BigQuery.
- Rodar `vigiar` sem `base-referencia` em máquina nova alerta todas as CDs de uma vez.

Ao receber uma nova conversa

Ler este documento e o README, rodar `python -m radar doctor cds_gfw_fora_da_area` e `python -m pytest -q` para ver o estado, e confirmar com o dono antes de qualquer ação que envie mensagem, grave estado em produção, agende tarefas ou publique código.